

PATENT APPLICATION CONTINUATION-IN-PART

TEMPORAL DIMENSION AUGMENTATION AND MULTI-MODAL MANIFOLD NAVIGATION FOR TIMESTAMPED EMBEDDING SPACES

Inventor: Eric G. Suchanek, PhD
Assignee: Flux-Frontiers
Filing Date: 2026-03-27
Application Type: Continuation-in-Part (CIP)
Parent Application: U.S. Patent Application [PARENT APPLICATION NUMBER]
Parent Filed: 2026-03-26

Contents

CROSS-REFERENCE TO RELATED APPLICATIONS

This application is a continuation-in-part of U.S. Patent Application [PARENT APPLICATION NUMBER], filed 2026-03-26, titled *System and Method for Offline Semantic Pre-Computation and Knowledge Graph Construction from Timestamped Text Corpora*, the entire disclosure of which is incorporated herein by reference. The parent application discloses and claims a system and method for offline semantic pre-computation of timestamped text corpora and construction of a hybrid knowledge graph therefrom.

The present continuation-in-part application discloses and claims subject matter not disclosed in the parent application, specifically: (1) configurable temporal dimension augmentation of pre-computed embedding vectors; (2) the `augment_with_time` algorithm and its parameterization; (3) multi-modal manifold navigation with time primitives `expand_dim()`, `orient_in_time()`, and `orient_toward()`; (4) three distinct flight modes (semantic, temporal, mixed) supported by a `TemporalFlyer` navigator; and (5) the generalization of scalar-dimension augmentation to arbitrary navigable axes beyond time.

All elements of the parent application's pipeline — including corpus ingestion, multi-phase NLP transformation, knowledge graph construction, multi-process parallel embedding, and JSON embedding cache — are incorporated herein by reference and serve as the foundation upon which the present invention operates.

FIELD OF THE INVENTION

The present invention relates to geometric navigation of high-dimensional embedding spaces, and more particularly to a method and system for augmenting pre-computed embedding vectors with a scaled temporal coordinate to create a unified $(D + 1)$ -dimensional space in which time is a first-class navigable geometric axis, and for navigating that space using multi-modal manifold flight primitives that support continuous interpolation between semantic and temporal traversal of any timestamped corpus.

BACKGROUND OF THE INVENTION

Standard sentence-transformer models produce fixed-dimensional real-valued vectors that encode semantic meaning. Two documents discussing similar topics will land close together in embedding space regardless of when they were written. A diary entry from 1660 and one from 1669 may occupy nearly identical coordinates if their semantic content is sufficiently similar. The temporal dimension is discarded entirely.

Prior art approaches to time-aware embedding include: (a) training specialized temporal language models from scratch, which requires large datasets and significant compute; (b) fine-tuning existing models with temporal supervision signals, which alters the semantic structure of the embedding space; (c) treating time as a metadata

filter applied after nearest-neighbor retrieval, which does not expose time as a navigable geometric direction; and (d) concatenating raw Unix timestamps or year integers directly to embedding vectors, which produces gross magnitude imbalance because raw time values ($\sim 10^9$ for Unix epoch) dwarf normalized embedding magnitudes ($\sim \mathcal{O}(1)$), causing the temporal dimension to dominate retrieval entirely.

None of the prior art provides: a principled, magnitude-preserving method for appending a scalar temporal coordinate as a $(D + 1)$ -th embedding dimension; a configurable weight parameter that continuously interpolates from temporally-neutral to temporally-dominant embedding spaces; or a manifold navigator equipped with time-specific primitives that can face and fly along the temporal axis, the semantic axes, or any blend thereof.

The parent application (incorporated herein by reference) describes a system that pre-computes 768-dimensional embedding vectors for every chunk in a timestamped corpus and stores them in an aligned JSON cache. The present invention builds upon this cache to create a navigable temporally-augmented space.

SUMMARY OF THE INVENTION

The present invention provides a method and system for temporal dimension augmentation of embedding vectors and multi-modal manifold navigation of the resulting augmented space.

The augmentation method appends a scalar temporal coordinate to each D -dimensional embedding vector to produce a $(D + 1)$ -dimensional vector. The temporal coordinate is first z-score normalized across the corpus, then scaled to match the mean L_2 norm of the embedding vectors, multiplied by a configurable weight parameter α . This produces a unified embedding space in which the temporal axis has a magnitude contribution commensurate with a single typical embedding axis when $\alpha = 1$, smaller when $\alpha < 1$, and larger when $\alpha > 1$.

The navigation system builds a k -nearest-neighbor graph over the augmented $(D + 1)$ -dimensional space and wraps the `TurtleND` N -dimensional manifold navigator from the `proteusPy` library with three new time primitives: `expand_dim()` to grow the navigator's basis by one dimension; `orient_in_time()` to rotate the navigator's heading to face the temporal axis; and `orient_toward()` to rotate the heading toward an arbitrary direction vector.

Three flight modes are defined: *semantic flight*, in which the navigator steers toward the semantically most-distant reachable node; *temporal flight*, in which the heading is oriented along the temporal axis and the navigator advances through purely chronological order; and *mixed flight*, in which the heading blends semantic and temporal directions via a configurable `time_blend` parameter on the interval $[0.0, 1.0]$.

The generalization principle: the augmentation and navigation machinery is not specific to time. Any scalar quantity — sentiment score, reading level, geographic

coordinate, market volatility, author index, topic cluster centroid — can be appended as an extra navigable axis using the same algorithm and the same primitives.

The invention has been reduced to practice and validated on a 8,413-entry corpus producing 769-dimensional augmented embeddings (768 semantic + 1 temporal) with a KNN graph at $k = 10$.

1. DETAILED DESCRIPTION OF THE PREFERRED EMBODIMENTS

1.1. System Overview and Relationship to Parent Application

The present invention receives as input the JSON embedding cache produced by the multi-process parallel embedder described in the parent application. That cache contains three index-aligned arrays:

- **embeddings**: an $N \times D$ float32 array of pre-computed sentence-transformer embeddings, where N is the number of corpus chunks and $D = 768$ in the preferred embodiment using the `all-mpnet-base-v2` model;
- **texts**: a length- N array of the corresponding chunk text strings;
- **timestamps**: a length- N array of ISO-format timestamp strings written directly from source metadata during the corpus emission stage described in the parent application.

The present invention converts the ISO timestamps to floating-point values (e.g., fractional years or Unix epoch seconds), augments the $N \times D$ embedding matrix to $N \times (D + 1)$ using the `augment_with_time` algorithm described below, constructs a k -nearest-neighbor graph over the augmented space, and provides the `TemporalFlyer` navigator for multi-modal traversal.

No modifications to the parent application’s pipeline are required. The augmentation is applied entirely in memory, post-cache, without modifying the stored embeddings.

1.2. Configurable Temporal Dimension Augmentation

1.2.1. The Problem of Magnitude Imbalance

A 768-dimensional sentence-transformer embedding vector has a mean L_2 norm of approximately 3–8 units in normalized practice. A raw timestamp expressed as, for example, a Unix epoch integer has a magnitude on the order of $10^9 - 10^{10}$. Appending this raw value as a 769th dimension would cause the temporal component to dominate all nearest-neighbor computations, collapsing the semantic structure of the space. Conversely, expressing timestamps as year integers (1660–1669 for the Pepys corpus) still produces values orders of magnitude different from the embedding norm without scaling.

The invention solves this problem through magnitude-preserving normalization.

1.2.2. The augment-with-time Algorithm

Let $\mathbf{E} \in \mathbb{R}^{N \times D}$ denote the embedding matrix and let $\mathbf{t} \in \mathbb{R}^N$ denote the vector of scalar time values (e.g., fractional year, Unix epoch, or any monotone temporal encoding).

The algorithm proceeds in three steps:

Step 1 — Z-score normalization of time values.

$$\tilde{t}_i = \frac{t_i - \bar{t}}{\sigma_t} \quad (1)$$

where $\bar{t}$ is the mean and σ_t is the standard deviation of the time values across the corpus. This centers and scales the temporal coordinate to unit variance, removing dependence on the choice of time encoding units.

Step 2 — Scaling to match embedding magnitude.

$$\hat{t}_i = \tilde{t}_i \cdot \alpha \cdot \frac{\|\bar{\mathbf{e}}\|}{\sqrt{D}} \quad (2)$$

where $\|\bar{\mathbf{e}}\|$ is the mean L_2 norm of the embedding rows (i.e., $\frac{1}{N} \sum_{i=1}^N \|\mathbf{e}_i\|_2$), D is the embedding dimension, and α is the configurable temporal weight parameter.

The factor $\|\bar{\mathbf{e}}\|/\sqrt{D}$ represents the expected per-axis contribution of a single embedding dimension. Multiplying by this factor ensures that $\hat{t}$ has the same magnitude as one typical embedding axis when $\alpha = 1$.

Step 3 — Column concatenation.

$$\mathbf{Z} = [\mathbf{E} \mid \hat{\mathbf{t}}] \in \mathbb{R}^{N \times (D+1)} \quad (3)$$

The augmented matrix $\mathbf{Z}$ replaces $\mathbf{E}$ for all downstream computations including KNN graph construction and manifold navigation.

1.2.3. The Alpha Parameter

The weight parameter $\alpha \geq 0$ provides continuous control over the relative influence of the temporal dimension:

- $\alpha = 0$: the temporal column is zeroed; $\mathbf{Z}$ reduces to $\mathbf{E}$ with an appended zero; temporal ordering has no influence on navigation.
- $\alpha = 1$: the temporal dimension contributes as much as one typical semantic axis; semantic and temporal structure are balanced.
- $\alpha > 1$: the temporal dimension is amplified above the average semantic axis; temporal proximity begins to dominate nearest-neighbor structure.

In the reference implementation, $\alpha = 1.0$ was used for all flight experiments described in Section ??.

1.2.4. *Reference Implementation*

Figure ?? shows the complete reference implementation of the `augment_with_time` function.

```

1 import numpy as np
2 from math import sqrt
3
4
5 def augment_with_time(
6     embeddings: np.ndarray,
7     time_values: np.ndarray,
8     alpha: float = 1.0,
9 ) -> np.ndarray:
10     """Append a scaled temporal coordinate as a (D+1)-th
11         dimension.
12
13     Args:
14         embeddings: (N, D) float32 array of embedding
15                     vectors.
16         time_values: (N,) float array of scalar time
17                     values in any
18                     monotone encoding (fractional year,
19                     Unix epoch,
20                     ordinal day, etc.).
21         alpha: Temporal weight parameter. alpha=0
22                recovers the
23                original embedding space; alpha=1
24                makes the
25                temporal axis contribute as much as
26                one typical
27                embedding dimension; alpha>1 makes
28                time dominant.
29
30     Returns:
31         (N, D+1) float32 array with the scaled temporal
32         coordinate
33         appended as the final column.
34     """
35     # Step 1: z-score normalize the time values across the
36     # corpus
37     t_mean = time_values.mean()
38     t_std = time_values.std()
39     t_norm = (time_values - t_mean) / t_std #
40         zero mean, unit var
41
42     # Step 2: scale to match the per-axis embedding
43     # magnitude
44     # mean_norm / sqrt(D) is the expected contribution of
45     # one axis
46     mean_norm = np.linalg.norm(embeddings, axis=1).mean()
47     D = embeddings.shape[1]
48     t_scaled = t_norm * alpha * (mean_norm / sqrt(D))
49
50     # Step 3: append the temporal coordinate as the
51     # (D+1)-th column
52     return np.column_stack([embeddings, t_scaled])

```

1.3. KNN Graph Construction over Augmented Space

Following augmentation, the invention constructs a k -nearest-neighbor graph over the rows of $\mathbf{Z}$. In the preferred embodiment, $k = 10$ and cosine distance is used as the similarity metric. Each node i in the graph has edges to its k nearest neighbors in the $(D + 1)$ -dimensional augmented space.

Because the temporal coordinate is part of the ambient space during KNN construction, two nodes that are semantically similar but temporally distant will be slightly farther apart in augmented space than in the original D -dimensional space, and two nodes that are semantically moderate but temporally close will be slightly nearer. The degree of this effect is controlled by α .

The KNN graph is the sole data structure consumed by the **TemporalFlyer** navigator. It does not require rebuilding when α is changed, provided the graph was constructed at the desired α value.

1.4. Time Navigation Primitives

The invention extends the **TurtleND** N -dimensional manifold navigator from the **proteusPy** library with three new primitives. **TurtleND** models a navigator in $\mathbb{R}^N$ as a position vector and an orthonormal frame (heading, left, up, and higher-dimensional analogues). It advances by stepping to adjacent nodes in the KNN graph.

1.4.1. *expand_dim()* — *Grow the Basis by One Dimension*

Problem: **TurtleND** is initialized with an N -dimensional basis. When the embedding space is augmented from D to $D + 1$ dimensions, the navigator’s internal basis must be extended to accommodate the new axis.

Solution: `expand_dim()` appends a new standard basis vector $\mathbf{e}_{D+1}$ (a zero vector with a 1 in the $(D + 1)$ -th position) to the navigator’s orthonormal frame. The existing D basis vectors are unchanged. No re-orthogonalization is required because $\mathbf{e}_{D+1}$ is by construction orthogonal to all vectors in the original D -dimensional subspace.

This primitive enables the navigator to be initialized on the original D -dimensional space and then promoted to the augmented $(D + 1)$ -dimensional space without rebuilding the navigator from scratch.

1.4.2. *orient_in_time()* — *Rotate Heading to Face the Temporal Axis*

Problem: After `expand_dim()`, the navigator knows the temporal axis exists, but its heading vector points in an arbitrary semantic direction. To perform temporal flight, the navigator must face forward in time.

Solution: `orient_in_time()` sets the navigator’s heading to $\mathbf{e}_{D+1}$ (the positive temporal axis direction) or its negation $-\mathbf{e}_{D+1}$ (backward in time). The remaining basis vectors are re-orthogonalized via Gram-Schmidt to maintain the orthonormal frame invariant.

After calling `orient_in_time()`, stepping forward in the KNN graph advances the navigator preferentially toward nodes with later timestamps, because the KNN graph was constructed in the augmented space where temporal proximity is encoded in the metric.

1.4.3. *orient_toward()* — *Rotate Heading Toward an Arbitrary Direction*

Problem: Semantic flight requires the navigator to face the semantically most-distant reachable node, while mixed flight requires a heading that blends temporal and semantic directions. Neither direction is a standard basis vector.

Solution: `orient_toward()` accepts an arbitrary direction vector $\mathbf{v} \in \mathbb{R}^{D+1}$ and rotates the navigator’s heading toward $\hat{\mathbf{v}} = \mathbf{v}/\|\mathbf{v}\|$ via a Householder reflection that maps the current heading to $\hat{\mathbf{v}}$ while preserving the orthonormality of the remaining basis vectors. The remaining frame vectors are updated by the same reflection.

This primitive is the common implementation substrate for both semantic and mixed flight modes.

1.5. The TemporalFlyer: Three Flight Modes

The `TemporalFlyer` class composes the augmented embedding matrix, the KNN graph, and the navigation primitives into a complete manifold navigation system. It maintains internal state including the current position index in the corpus, the navigator’s heading, and the flight log (sequence of visited node indices and their timestamps).

1.5.1. *Semantic Flight*

In semantic flight, the navigator seeks the semantically most-distant entry at each step:

1. From the current node i , identify the k KNN neighbors.
2. Compute the augmented-space direction vector from the current position to each neighbor.
3. Select the neighbor whose direction vector has the smallest cosine similarity with the current heading (i.e., is most orthogonal or opposed), weighted by distance. This steers the navigator toward unexplored semantic territory.
4. Call `orient_toward()` with the selected direction, then advance to the selected neighbor.

In semantic flight, time is incidental. The navigator follows meaning; temporal progression emerges only to the extent that semantically distant entries happen to be temporally distant. In the reference experiment, semantic flight produced a Kendall $\tau = +0.19$, indicating weak forward temporal drift consistent with a corpus whose semantic content changes gradually over time.

1.5.2. Temporal Flight

In temporal flight, the navigator orients along the temporal axis and follows it as directly as the KNN graph permits:

1. Call `orient_in_time()` to set the heading to $\mathbf{e}_{D+1}$.
2. At each step, select the KNN neighbor with the maximum projection onto the current heading direction (i.e., the neighbor most forward in augmented-space time direction).
3. Advance to the selected neighbor without changing the heading.

Because the KNN graph was constructed in the augmented space, the neighbors most forward in the temporal direction are nodes that are both temporally close and semantically proximate. The navigator does not simply traverse entries in timestamp order; it follows the manifold’s geometry, which is curved relative to the temporal axis. This produces shorter mean time steps (0.61 years per step in the reference experiment) than semantic flight (1.56 years per step), with a Kendall $\tau = -0.38$ indicating that the path is locally coherent in time but globally non-monotone, as expected for a curved manifold.

1.5.3. Mixed Flight

Mixed flight continuously interpolates between semantic and temporal directions using a configurable `time.blend` parameter $\beta \in [0.0, 1.0]$:

1. Compute the semantic direction $\mathbf{d}_s$ (direction to the most semantically interesting neighbor, as in semantic flight).
2. Compute the temporal direction $\mathbf{d}_t = \mathbf{e}_{D+1}$.
3. Compute the blended heading:

$$\mathbf{d}_{\text{mixed}} = \frac{(1 - \beta) \mathbf{d}_s + \beta \mathbf{d}_t}{\|(1 - \beta) \mathbf{d}_s + \beta \mathbf{d}_t\|} \quad (4)$$

4. Call `orient_toward()( $\mathbf{d}_{\text{mixed}}$ )` and advance to the neighbor with the maximum projection onto the blended heading.

At $\beta = 0$, mixed flight is identical to semantic flight. At $\beta = 1$, it is identical to temporal flight. Intermediate values produce paths that spiral through both time and meaning simultaneously, covering broader temporal spans (2.78 years total in the reference experiment at $\beta = 0.5$) with intermediate time coherence (Kendall $\tau = -0.15$).

1.6. Measured Results

1.6.1. Experimental Configuration

All results were obtained on the Pepys corpus as prepared by the parent application’s pipeline.

Parameter	Value
Corpus entries	8,413
Embedding model	all-mpnet-base-v2
Semantic embedding dim.	768
Augmented dim. ($D + 1$)	769
Temporal weight α	1.0
KNN graph degree k	10
Similarity metric	Cosine
Hardware	Apple Silicon (CPU only, no GPU)

Flight experiment parameters:

Parameter	Value
Origin timestamp	1663-10-21
Destination timestamp	1664-01-23
Maximum steps	150

1.6.2. Flight Results

Mode	Path Length	Monotonicity	Kendall τ	Mean Δt (yr)	Total Span (yr)
Semantic	79 steps	0.54	+0.19	1.56	2.0
Temporal	151 steps	0.52	-0.38	0.61	2.3
Mixed ($\beta = 0.5$)	142 steps	0.52	-0.15	1.13	2.7

Interpretation.

- **Semantic flight** ($\tau = +0.19$): weakly forward in time. The navigator follows meaning; temporal progression is incidental. The corpus’s semantic structure drifts slightly forward over the 9-year span, explaining the positive τ .
- **Temporal flight** ($\tau = -0.38$): shorter time steps (0.61 yr/step) and a longer path (151 steps). The negative τ indicates the navigator is following the manifold’s geometric curvature, not a monotone timestamp sequence. The path stays temporally local at each hop but traverses the manifold in a non-monotone global order, confirming that the temporal axis is genuinely orthogonal to some semantic axes in the augmented space.
- **Mixed flight** ($\beta = 0.5$, $\tau = -0.15$): intermediate behavior. Longer mean time steps than temporal (1.13 vs. 0.61 yr/step), broader total temporal coverage (2.78 yr), and moderate time coherence. The navigator balances exploration of semantic space with temporal progression.

1.6.3. Manifold Geometry Context

The reference corpus occupies a 768-dimensional ambient space but has TwoNN intrinsic dimensionality of 14.26 (reported in the parent application). The effective manifold is low-dimensional ($\sim 14D$) and curved. This curvature explains why temporal flight produces a non-monotone Kendall τ : the manifold’s geometry does not align with the global time axis, confirming that the temporal augmentation reveals genuine geometric structure rather than trivially reordering the corpus by timestamp.

1.7. Generalization to Arbitrary Scalar Dimensions

The `augment_with_time` algorithm, the `expand_dim()` primitive, the `orient_toward()` primitive, and the three flight modes are not specific to time. The same machinery applies to any scalar quantity that can be computed for each corpus entry and carries meaningful ordering or magnitude information.

Examples of directly applicable scalar dimensions:

Scalar Quantity	Navigation Capability
Sentiment score	Navigate from negative to positive affect
Reading level (Flesch-Kincaid)	Navigate from simple to complex prose
Geographic latitude/longitude	Navigate through spatial distribution
Market volatility index	Navigate from stable to turbulent periods
Document age	Navigate from recent to archival content
Author index	Navigate between author styles
Topic cluster centroid distance	Navigate toward/away from a topic cluster
Citation count	Navigate from obscure to highly-cited works

In each case, the procedure is identical to the temporal case: compute the scalar values, call `augment_with_time` with appropriate α , build or rebuild the KNN graph, initialize a `TemporalFlyer` (or analogous navigator), and use `orient_toward()` with the $(D + 1)$ -th standard basis vector as the target direction.

Multiple scalar dimensions can be appended simultaneously by applying `augment_with_time` iteratively. Each application increases the embedding dimension by one and adds an independent navigable axis. The navigation primitives generalize to $D + k$ dimensions for k appended scalars.

Figure ?? shows the generalized call pattern for multi-scalar augmentation.

```

1 def augment_with_scalar(
2     embeddings: np.ndarray,
3     scalar_values: np.ndarray,
4     alpha: float = 1.0,
5 ) -> np.ndarray:
6     """Append any scalar quantity as a navigable (D+1)-th
7         dimension.
8
9     Identical to augment_with_time() but accepts arbitrary
10        scalar values.
11    May be applied iteratively to append multiple axes.
12
13    Example --- append sentiment and time simultaneously:
14
15        Z = augment_with_scalar(embeddings,
16                                sentiment_scores, alpha=0.5)
17        Z = augment_with_scalar(Z, time_values, alpha=1.0)
18        # Z is now (N, D+2): semantic + sentiment +
19            temporal axes
20
21    Args:
22        embeddings:      (N, D) float32 embedding matrix.
23        scalar_values:   (N,) array of scalar values in any
24                        units.
25        alpha:           Weight parameter (0=no effect,
26                        1=one axis,
27                        >1=amplified).
28
29    Returns:
30        (N, D+1) float32 augmented matrix.
31    """
32    s_mean = scalar_values.mean()
33    s_std = scalar_values.std()
34    s_norm = (scalar_values - s_mean) / s_std
35
36    mean_norm = np.linalg.norm(embeddings, axis=1).mean()
37    D = embeddings.shape[1]
38    s_scaled = s_norm * alpha * (mean_norm / sqrt(D))
39
40    return np.column_stack([embeddings, s_scaled])

```

Listing 2: Generalized scalar-dimension augmentation — iterative application to append multiple navigable axes.

1.8. System Architecture Summary

Figure ?? summarizes the complete system, including the parent application’s pipeline stages (shown in boxes with dashed borders) and the present invention’s additions (shown in boxes with solid borders).

```

PARENT APPLICATION STAGES (incorporated by reference)
-----
Stage 1: DiaryTransformer -- NLP enrichment, chunking,
        topic/context classification
Stage 2: Corpus emission -- .md chunks with YAML
        frontmatter,
        direct temporal provenance write
Stage 3: DocKG SQLite build -- structural graph, BM25
Stage 4: DocKG LanceDB build -- 768-dim vector index
Stage 5: Multi-process Embedder -- parallel
        sentence-transformer
Stage 6: JSON Cache -- aligned {embeddings, texts,
        timestamps}

PRESENT INVENTION (CIP additions)
-----
Step A:  Parse timestamps from JSON cache -> float
        time_values
Step B:  augment_with_time(embeddings, time_values,
        alpha)
        -> Z: (N, 769) augmented matrix
Step C:  Build KNN graph over Z (k=10, cosine distance)
Step D:  Initialize TemporalFlyer with KNN graph and Z
        - expand_dim()          : extend TurtleND basis to
          D+1
        - orient_in_time()      : face the temporal axis
        - orient_toward(v)     : face arbitrary direction
Step E:  Execute flight (semantic | temporal | mixed)
        -> path: [(index, timestamp, text), ...]
    
```

Listing 3: Full pipeline including parent application stages and CIP additions.

2. CLAIMS

What is claimed is:

INDEPENDENT CLAIMS

Claim 1. A computer-implemented method for augmenting pre-computed embedding vectors with a temporal coordinate, comprising:

- (a) receiving a matrix $\mathbf{E} \in \mathbb{R}^{N \times D}$ of pre-computed floating-point embedding vectors, each row corresponding to a text chunk from a timestamped corpus, wherein the embedding vectors were computed by a pre-trained sentence-transformer model;
- (b) receiving a vector $\mathbf{t} \in \mathbb{R}^N$ of scalar time values corresponding index-for-index to the rows of $\mathbf{E}$;
- (c) computing z-score normalized time values: $\tilde{t}_i = (t_i - \bar{t}) / \sigma_t$, where $\bar{t}$ is the mean and σ_t is the standard deviation of $\mathbf{t}$;
- (d) computing a scaled temporal coordinate: $\hat{t}_i = \tilde{t}_i \cdot \alpha \cdot \|\bar{\mathbf{e}}\| / \sqrt{D}$, where $\|\bar{\mathbf{e}}\|$ is the mean L_2 row norm of $\mathbf{E}$, D is the embedding dimension, and α is a configurable weight parameter;
- (e) forming the augmented matrix: $\mathbf{Z} = [\mathbf{E} \mid \hat{\mathbf{t}}] \in \mathbb{R}^{N \times (D+1)}$ by appending $\hat{\mathbf{t}}$ as the $(D + 1)$ -th column;

wherein, when $\alpha = 1$, the magnitude of the appended temporal column is commensurate with the expected per-axis magnitude of a single dimension of $\mathbf{E}$, so that temporal and semantic proximity are weighted equally in subsequent nearest-neighbor computations over $\mathbf{Z}$.

Claim 2. The method of Claim 1, further comprising:

- (a) constructing a k -nearest-neighbor graph over the rows of the augmented matrix $\mathbf{Z}$, wherein each node corresponds to a corpus chunk and each edge connects a node to one of its k nearest neighbors under cosine distance in the $(D + 1)$ -dimensional augmented space;
- (b) initializing an N -dimensional manifold navigator with a position vector and orthonormal basis frame;
- (c) extending the navigator's basis from D dimensions to $D + 1$ dimensions by appending a new standard basis vector $\mathbf{e}_{D+1}$ without modifying the existing D basis vectors;
- (d) executing a multi-step traversal of the k -nearest-neighbor graph that comprises at least one of:
 - (i) orienting the navigator's heading to face the $(D + 1)$ -th axis direction $\mathbf{e}_{D+1}$ and advancing preferentially toward neighbors with the greatest projection onto that heading, thereby performing temporal flight;
 - (ii) orienting the heading toward a semantically distant neighbor direction and advancing preferentially toward nodes maximizing semantic distance from the current position, thereby performing semantic flight;
 - (iii) blending temporal and semantic heading directions using a configurable weight parameter $\beta \in [0, 1]$ and advancing toward the neighbor with the greatest projection onto the blended heading, thereby performing mixed flight;
- (e) recording the sequence of visited nodes and their associated timestamps as a flight path.

Claim 3. A computer-implemented system for multi-modal manifold navigation of a temporally-augmented embedding space, comprising:

- (a) an augmentation module configured to receive a D -dimensional embedding matrix $\mathbf{E}$ and a vector of scalar time values $\mathbf{t}$, apply z-score normalization to $\mathbf{t}$, scale the normalized values to match the mean per-axis magnitude of $\mathbf{E}$ multiplied by a configurable weight parameter α , and return the $(D + 1)$ -dimensional augmented matrix $\mathbf{Z}$;
- (b) a graph builder module configured to construct a k -nearest-neighbor graph over the rows of $\mathbf{Z}$ using cosine distance;
- (c) a manifold navigator equipped with:
 - (i) a basis extension primitive that grows the navigator's orthonormal frame from D to $D + 1$ dimensions by appending the standard basis vector $\mathbf{e}_{D+1}$;
 - (ii) a temporal orientation primitive that rotates the navigator's heading to face $\pm \mathbf{e}_{D+1}$ and re-orthogonalizes the remaining frame vectors;
 - (iii) a directed orientation primitive that rotates the navigator's heading toward any specified direction vector via Householder reflection while preserving frame orthonormality;
- (d) a flight executor configured to execute at least three navigation modes — semantic flight, temporal flight, and mixed flight — over the k -nearest-neighbor graph using the manifold navigator's heading and orientation primitives;
- (e) wherein the system is configured to receive as input the JSON embedding cache produced by the offline pre-computation pipeline of U.S. Patent Application [PARENT APPLICATION NUMBER], without modification to that pipeline.

Claim 4. A computer-implemented method for scalar dimension augmentation and navigation of embedding spaces, comprising:

- (a) receiving a D -dimensional embedding matrix $\mathbf{E} \in \mathbb{R}^{N \times D}$ and a vector of scalar values $\mathbf{s} \in \mathbb{R}^N$, wherein each scalar value encodes a property of the corresponding corpus entry other than semantic content;
- (b) applying the augmentation method of Claim 1 with $\mathbf{s}$ substituted for $\mathbf{t}$ to form a $(D + 1)$ -dimensional augmented matrix;
- (c) constructing a k -nearest-neighbor graph over the augmented matrix;
- (d) navigating the k -nearest-neighbor graph using the manifold navigator of Claim 3 by orienting the heading along the $(D + 1)$ -th axis direction to traverse the corpus in order of increasing scalar value;
- (e) wherein the scalar property is selected from the group consisting of: sentiment score, reading level score, geographic coordinate, market volatility index, document age, author identifier, topic cluster centroid distance, citation count, and any other scalar quantity computable at ingest time without neural network inference.

DEPENDENT CLAIMS

Claim 5. The method of Claim 1, wherein $\alpha = 0$ recovers the original D -dimensional embedding space as a degenerate case of the $(D + 1)$ -dimensional augmented space in

which the temporal column is identically zero, enabling the augmentation module to be present in the pipeline without effect when temporal navigation is not desired.

Claim 6. The method of Claim 1, wherein $\alpha > 1$ causes the temporal dimension to contribute more than a single typical semantic axis, progressively restricting the KNN graph’s neighborhood structure to temporally proximate entries, and wherein $\alpha = 1$ is the preferred embodiment that balances semantic and temporal proximity in the KNN metric.

Claim 7. The method of Claim 2, wherein temporal flight produces a Kendall rank correlation coefficient $\tau < 0$ between the step index along the flight path and the timestamp at each step, indicating that the navigator is traversing the curved manifold geometry rather than simply sorting the corpus by timestamp, and wherein this non-monotone behavior is a measurable characteristic of the manifold’s curvature relative to the temporal axis.

Claim 8. The method of Claim 2, wherein mixed flight with $\beta = 0.5$ produces a total temporal span greater than either pure semantic flight or pure temporal flight over the same maximum number of steps, reflecting the navigator’s ability to explore a broader region of the curved manifold by blending orthogonal geometric directions.

Claim 9. The system of Claim 3, wherein the manifold navigator is implemented as the `TurtleND` class from the `proteusPy` library, and wherein the basis extension primitive, temporal orientation primitive, and directed orientation primitive are new methods added to `TurtleND` that do not modify any existing functionality of that class.

Claim 10. The system of Claim 3, wherein the k -nearest-neighbor graph is constructed with $k = 10$ and cosine distance over a 769-dimensional augmented space comprising 768-dimensional `all-mpnet-base-v2` embeddings augmented with one temporal coordinate at $\alpha = 1.0$, for a corpus of approximately 8,413 entries.

Claim 11. The method of Claim 2, further comprising measuring the temporal coherence of a flight path by computing the Kendall rank correlation coefficient between the sequence index and the timestamp sequence of the flight path, wherein: a coefficient near $+1.0$ indicates monotonically forward temporal traversal; a coefficient near 0 indicates temporally uncorrelated traversal; and a coefficient near -1.0 indicates highly locally-coherent but globally non-monotone traversal characteristic of manifold curvature.

Claim 12. The method of Claim 4, wherein the augmentation method of Claim 1 is applied iteratively with k distinct scalar value vectors to produce a $(D + k)$ -dimensional augmented matrix, each additional column encoding an independent navigable axis, and wherein the k -nearest-neighbor graph over the $(D + k)$ -dimensional space simultaneously encodes all k scalar properties in the neighborhood structure.

Claim 13. The method of Claim 1, wherein the scalar time values $\mathbf{t}$ are derived from ISO-format timestamps that were written directly to structured document metadata during the corpus emission stage of the pipeline of U.S. Patent Application [PARENT APPLICATION NUMBER], without any language model extraction, ensuring 100% temporal reliability regardless of the language model’s size or temporal reasoning capability.

Claim 14. The system of Claim 3, wherein the flight executor records, for each step of each flight path, at least: the corpus index of the visited node; the timestamp of the visited node; the text content of the visited node; and the blended heading direction used to select that node, enabling post-hoc analysis of the navigator’s trajectory through the augmented space.

Claim 15. The method of Claim 2, wherein the k -nearest-neighbor graph is reused unchanged across multiple flight executions with different origin nodes, destination nodes, or flight modes, such that the $\mathcal{O}(N \cdot D)$ graph construction cost is paid once per $(D + 1)$ -dimensional augmentation configuration and amortized across all subsequent flights.

ABSTRACT

A method and system for appending a scalar temporal coordinate to pre-computed embedding vectors as a configurable $(D + 1)$ -th dimension, enabling time to function as a first-class navigable geometric axis in embedding space. The augmentation algorithm z-score normalizes the temporal values, scales them to match the mean per-axis magnitude of the embedding vectors via the factor $\alpha \cdot \|\bar{\mathbf{e}}\|/\sqrt{D}$, and appends the result as the final column of the embedding matrix. A configurable weight parameter α continuously controls the relative influence of the temporal dimension from neutral ($\alpha = 0$) through balanced ($\alpha = 1$) to temporally dominant ($\alpha > 1$). A k -nearest-neighbor graph constructed over the augmented space serves a **TemporalFlyer** manifold navigator equipped with three time primitives: `expand_dim()` (extend the navigator’s basis), `orient_in_time()` (face the temporal axis), and `orient_toward()` (face an arbitrary direction). Three flight modes are defined: semantic flight (navigate by meaning), temporal flight (navigate by time), and mixed flight (blend both via a `time_blend` parameter). Validated on a 8,413-entry corpus in 769-dimensional augmented space ($k = 10$, $\alpha = 1.0$), the system produces measurably distinct navigation behaviors: temporal flight achieves shorter mean time steps (0.61 yr/step) with Kendall $\tau = -0.38$, reflecting manifold curvature; semantic flight produces longer steps (1.56 yr/step) with $\tau = +0.19$; and mixed flight (blend = 0.5) achieves the broadest temporal coverage (2.78 yr total). The augmentation and navigation pattern generalizes to any scalar quantity (sentiment, reading level, geographic coordinate, market volatility), enabling arbitrary navigable axes without architectural changes. The system receives as input the JSON embedding cache produced by the offline pre-computation pipeline disclosed in the parent application, requiring no modification to that pipeline.

DRAWINGS AND FIGURES

The following figures are included in this specification:

- **Figure 1** (Listing ??): Reference implementation of `augment_with_time` showing the three-step algorithm: z-score normalization, magnitude-preserving scaling, and column concatenation.
- **Figure 2** (Listing ??): Generalized scalar-dimension augmentation function demonstrating iterative application for multiple navigable axes.
- **Figure 3** (Listing ??): Full pipeline diagram showing the relationship between the parent application's six pipeline stages and the five new steps of the present invention.

Respectfully submitted,

Eric G. Suchanek, PhD

Inventor

Flux-Frontiers

Filing Date: 2026-03-27